Making Real-Life JARVIS — Episode 6: Autonomous Website Control

by Pulse of AI • Created by Swayam Dhawale

■■ **PREREQUISITE NOTICE:** This series builds incrementally. If you have not configured the dependencies and API keys from previous episodes (Parts 1 to 5), this script might not run. Catch up on previous parts below!

1. Previous Episodes Series Directory

Episode	Focus & Description	Watch Link
Part 1	JARVIS AI Core Initialization in 60s	Watch Episode 1
Part 2	Voice Pipeline Setup & Robotic Tone Fixes	Watch Episode 2
Part 3	Gemini Live Multimodal Brain Migration	Watch Episode 3
Part 4	Real-Time Bi-Directional Audio Streaming	Watch Episode 4
Part 5	70+ Global Languages & Real-Time Switching	Watch Episode 5

2. Episode 6 Overview & Setup Steps

- **Step 1:** Install requirements: pip install google-genai pyaudio python-dotenv
- **Step 2:** Add your API Key inside .env: GOOGLE_API_KEY=your_key_here
- **Step 3:** Save the code as jarvis_ep6.py and execute with: python jarvis_ep6.py

3. Source Code (jarvis_ep6.py)

```
import asyncio, datetime, os, sys, webbrowser, pyaudio
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

FORMAT, CHANNELS, RATE_IN, RATE_OUT, CHUNK = pyaudio.paInt16, 1, 16000, 24000, 512
p = pyaudio.PyAudio()
model_is_speaking = False

def open_website(url: str) -> str:
    clean_url = url.strip()
    if not clean_url.startswith(("http://", "https://")):
        clean_url = "https://" + clean_url
    webbrowser.open(clean_url)
    return f"Opened {clean_url}"

tools = [open_website]

def get_time_greeting() -> str:
    h = datetime.datetime.now().hour
    if 5 <= h < 12: return "Good morning, sir."
    elif 12 <= h < 17: return "Good afternoon, sir."
    else: return "Good evening, sir."

async def send_audio(session):
    global model_is_speaking
    mic = p.open(format=FORMAT, channels=CHANNELS, rate=RATE_IN, input=True, frames_per_buffer=CHUNK)
    try:
        while True:
            data = await asyncio.to_thread(mic.read, CHUNK, exception_on_overflow=False)
            if model_is_speaking: continue
            await session.send_realtime_input(audio={"data": data, "mime_type": "audio/pcm;rate=16000"})
    finally:
        mic.stop_stream(); mic.close()
```

```
async def receive_audio(session, audio_queue):
    global model_is_speaking
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    resps = []
                    for call in response.tool_call.function_calls:
                        if call.name == "open_website":
                            u = call.args.get("url", "https://google.com")
                            res = open_website(u)
                            resps.append(types.FunctionResponse(name=call.name, id=call.id, response={"result": res}))
                    if resps: await session.send_tool_response(function_responses=resps)
                sc = response.server_content
                if sc is not None:
                    if sc.interrupted:
                        while not audio_queue.empty(): audio_queue.get_nowait()
                        model_is_speaking = False
                    if sc.model_turn is not None:
                        if not model_is_speaking: model_is_speaking = True
                        for part in sc.model_turn.parts:
                            if part.inline_data and part.inline_data.data:
                                await audio_queue.put(part.inline_data.data)
                    if sc.turn_complete: model_is_speaking = False
    finally: await audio_queue.put(None)

async def play_audio(audio_queue):
    spk = p.open(format=FORMAT, channels=CHANNELS, rate=RATE_OUT, output=True, frames_per_buffer=CHUNK)
    try:
        while True:
            d = await audio_queue.get()
            if d is None: break
            await asyncio.to_thread(spk.write, d)
    finally:
        spk.stop_stream(); spk.close()

async def main():
    greeting = get_time_greeting()
    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"], tools=tools,
        speech_config=types.SpeechConfig(voice_config=types.VoiceConfig(prebuilt_voice_config=types.PrebuiltVoiceConfig(voice_name="Puck"))),
        system_instruction=types.Content(parts=[types.Part.from_text(
            text="You are JARVIS, created by Swayam Dhawale, Founder of Pulse of AI. Calm and articulate tone like Tony Stark's JARVIS. Always address us
        )])
    )
    async with client.aio.live.connect(model="gemini-3.1-flash-live-preview", config=config) as session:
        q = asyncio.Queue()
        st = asyncio.create_task(send_audio(session))
        rt = asyncio.create_task(receive_audio(session, q))
        pt = asyncio.create_task(play_audio(q))
        await session.send_realtime_input(text=f"Say aloud: 'All protocols initialized. {greeting}'")
        try: await asyncio.gather(st, rt, pt)
        finally: st.cancel(); rt.cancel(); pt.cancel()

if __name__ == "__main__":
    if sys.platform == "win32": asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolicy())
    try: asyncio.run(main())
    finally: p.terminate()
```

Coming in Episode 7: Local Desktop App Launching (VS Code, Spotify, Photoshop)! Subscribe to Pulse of AI on YouTube so you do not miss the drop!